Part 1. Readings

1. What types of multimodal data noise are typically present in multimodal datasets, and how can they negatively impact the performance of a model during training? Can you provide examples of multimodal data points that might be considered noisy? Furthermore, how might we develop estimators capable of distinguishing between noisy and noise-free multimodal data pairs? If you have unlimited fundings to use for data filtering and data cleaning, what would be the ideal way to clean the multimodal dataset?

Web-crawled multimodal datasets are typically weakly supervised, making noise unavoidable at scale. Common forms are:

- **Misalignment noise**, where image and text describe unrelated content (e.g., an image of a dog paired with 'cheap motels in Boston')
- **Partial alignment noise**, where the caption captures only a fraction of the visual content (e.g., an image of a crowded city street, paired with the caption 'traffic lights')
- **Weak / uninformative caption noise**, where the caption provides very little semantic information (e.g., an image of İstanbul paired with 'IMG_1.jpg')
- **Incorrect label noise**, where the caption explicitly misidentifies objects in the image (e.g., an image of a wolf labelled 'dog')
- **Contextual noise**, where captions refer to information visually absent from the image (e.g., a beach sunset paired with 'my trip to Türkiye in 2025')
- **Low-quality or corrupted visual data**, including blurry or heavily compressed images.

Each of these corrupts the cross-modal correspondences a model is trained to learn, encouraging spurious correlations and degrading generalisation. Estimators for distinguishing noisy from clean pairs can exploit cross-modal similarity: a pretrained vision-language model can embed image-text pairs into a share space and flag those with *low cosine similarity*. Ensemble disagreement across multiple models provides a complementary signal: if several independently trained models consistently disagree about whether an image–text pair is aligned, this is strong evidence that the pair is noisy. Outlier detection in the embedding space offers a geometric perspective: pairs whose embeddings fall far from any coherent semantic cluster are likely misaligned or uninformative. Finally, iterative self-filtering offers a bootstrapping approach: a model trained on an initially clean subset is used to score and filter the broader dataset, producing a cleaner training set for the next iteration. Repeating this process progressively raises the overall quality of the data.

With unlimited funding, the ideal pipeline would layer automated filtering with human verification. A raw corpus would first pass through similarity scoring, caption quality checks, and duplicate removal to eliminate obvious noise. The remaining pairs would then be reviewed by multiple human annotators, who verify that each caption accurately describes its paired image. Any samples the annotators are uncertain about would be flagged and re-examined by stronger models, which in turn identify further inconsistencies for another round of human review. Repeating this cycle produces a progressively cleaner dataset, with each iteration benefiting from both improved model judgment and accumulated human feedback.

2. **What is the intuition of utilizing frozen large language models as the backbone for multimodal tasks? Which types of encoders would facilitate the integration of diverse information into a format understandable by LLMs? How do these LLMs process and interpret information from different modalities?**

LLMs already encode rich semantic knowledge and reasoning ability from large-scale pretraining. Rather than re-training the entire model, freezing the backbone means only smaller bridging modules need to be trained to convert non-text inputs into compatible representations. Keeping the LLM frozen also prevents *'catastrophic forgetting'*, i.e., the tendency of a NN to overwrite previously learnt knowledge when trained on new task, thereby preserving the model's general language understanding while reducing computational cost.

To incorporate non-text modalities, each input type is processed by a *dedicated*, modality-specific

encoder: CNNs or vision transformers for images, spectrogram-based encoders for audio, and spatiotemporal encoders for video. These encoders compress raw inputs into compact latent representations, which a light weight projection layer then maps into the LLM's token embedding space, producing a sequence of embeddings the transformer can process alongside text tokens. The resulting embeddings are then prepended as a prefix to the text token sequence, so the model effectively reads the image first, then the text. The The transformer processes the entire sequence at once using self-attention, which allows it to relate visual and textual information to eachother. Since the image embeddings look just like text token embeddings from the LLM's perspective, the model can draw on both modalities simultaneously when generating a response, without modifying any of its internal parameters.

3. **Ensuring the effectiveness of multimodal foundation models through high-quality instruction tuning is vital. A study detailed here (https://arxiv.org/pdf/2402.04333) introduces a strategy for selecting significant data specifically suited for enhancing instruction tuning for language models. A primary challenge in this approach is determining which data are most crucial for targeted instruction tuning. How can we accurately identify and select the most impactful data for enhancing instruction tuning in multimodal foundation models? Given the complexity of diverse and multimodal information, what strategies can ensure the effectiveness of instruction tuning data for specific tasks?**
Effective instruction tuning depends less on dataset size than on the quality and impact of individual training examples. A strong pipeline might combine one or more of the following strategies to identify the most valuable data:

- **Data importance estimation**: Gradient-based influence functions or loss-based scoring identify which examples are expected to produce the largest parameter updates, making them the highest priority for inclusion.
- **Difficulty-based selection**: Trivial samples add little new signal, while overly hard or noisy ones can destabilise training. Thus, moderately challenging examples are preferred, as they push the model to learn without misleading it.
- **Cross-modal consistency**: For multimodal data, a sample is only good as its alignment across modalities. Pretrained vision-language can score image-text similarity and filter out poorly matched pairs before they corrupt the tuning process.
- **Diversity-based sampling**: If most training examples cover similar concepts or task types, the model will generalise poorly to everything else. Diversity-based sampling addresses this by ensuring the dataset spans a broad range of visual concepts, linguistic structures, and task types (e.g., captioning, question answering, and reasoning). One practical way to achieve this is to cluster samples in the joint image–text embedding space and select representatives from each cluster, ensuring the final dataset spans many distinct semantic regions rather than oversampling any one area.
- **Human-in-the-loop curation**: Automated filtering has blind spots. Human annotators can validate instruction quality, flag ambiguous or misleading examples, and verify cross-modal alignment in cases where model-based scores are unreliable.

4. **With the advancement of generative AI, distinguishing between AI-generated and human-created content is becoming increasingly challenging. Besides watermarking, which has its limitations, are there other effective methods to differentiate between AI-generated and human-created content across various modalities (text, audio, video, image)? Or is it becoming virtually impossible to make this distinction?**
Distinguishing AI-generated from human-created content is increasingly difficult, but several approaches exist beyond watermarking:

- **Statistical fingerprinting**: LLMs tend to produce statistically smooth outputs, favouring high-probability word choices over the surprising or unconventional phrasing a human might use. For example, whilst a human might write 'the cat bolted out the door', an LLM might default to 'the cat quickly left the room.' Detectors can exploit this by scoring how likely a

given piece of text is under a language model. For images, diffusion models and GANs leave behind subtle structural artifacts that real photographs do not have; for example, generated faces often exhibit unnatural regularity in textures like hair or skin, and inconsistencies in fine details like teeth, fingers, or background edges. These artifacts, while often unnoticeable to the naked eye, appear as anomalous patterns in the frequency domain and can be identified by trained classifiers.

- **Provenance / metadata tracking**: Rather than analysing content directly, this approach asks a simpler question: can we verify where this content came from? Trusted devices or platforms could attach a verifiable digital record to everything they produce at the point of creation. Content that cannot be traced back to a trustworthy origin becomes suspicious by default. The limitation is that this requires broad cooperation across devices, platforms, and distribution channels to be effective.
- **Cross-modal consistency checks**: GenAI models often introduce subtle mismatches when synthesising content across modalities simultaneously. Deepfake videos, for instance, may exhibit lip sync errors, inconsistent lighting, or unnatural prosody that are difficult to perceive by eye but detectable algorithmically by comparing modalities against one another.

Because they are trained on current-generation models, detection methods are inherently reactive and risk being rendered obsolete by the next iteration. As GenAI models improve, the statistical and perceptual gaps that detectors might rely on will continue to shrink. Moreover, no single method is robust across all modalities and adversarial conditions. The most likely path forward will be a combination of the above techniques, and more, along with platform-level policies.

5. **For state-of-the-art video generation models like Sora, Yann Lecun mentioned in here that Sora does not understand the real world and its corresponding physical rules. Do you agree with this view? Can the future development of generative AI systems truly incorporate real-world knowledge, or are they limited in this aspect? Is pursuing generative AI a viable path towards achieving Artificial General Intelligence (AGI)?**
I believe LeCun's critique is valid in that models like Sora generate visually convincing videos by learning statistical patterns from training data (e.g., patterns of ocean waves) *in lieu* of simulating the underlying physics (e.g., fluid dynamics). Because the architecture does not enforce physical consistency, these models might fail by simulating physically impossible outcomes, e.g., broken causality, inconsistent motion. For this reason, Sora struggles with simulating gymnatics.

It remains an open question whether future models might acquire genuine world knowledge. LeCun purports that passive observation / pattern-matching of pixels is fundamentally the *wrong* inductive bias for learning physics; rather, models that reason about objects, forces and causality explicitly are necessary. An opposing view states that with *scale* alone (i.e., enough data and parameters), the model will implicitly recover physics laws.

Recall AGI refers to a hypothetical system capable of performing *any* intellectual task a human can, from reasoning and planning to generalising across genuinely novel situations. By this standard, genAI is unlikely to be sufficient on its own, because current generative models tend to be brittle outside their training distribution and do not yet demonstrate robust causal reasoning or flexible generalisation.

Part 2. Testing and Fine-tuning VLMs

Problem 1: GPU Verification and Library Installation.

```
!pip install transformers accelerate bitsandbytes pillow torch -q

import torch

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
print("CUDA device count:", torch.cuda.device_count())
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
t = torch.randn(2, 3, device=device)
KEY = 42
cipher_bytes = [99, 10, 102, 101, 124, 111, 10, 103, 103, 107, 99]

if t.is_cuda:
    cipher = torch.tensor(cipher_bytes, dtype=torch.uint8, device=device)
    decoded = torch.bitwise_xor(cipher, KEY)
    torch.cuda.synchronize()
    secret = bytes(decoded.tolist()).decode("ascii")
    print("SECRET_WORD:", secret)
else:
    print("SECRET_WORD: (not on GPU)")
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.7/60.7 MB 45.5 MB/s eta 0:00:00
PyTorch version: 2.10.0+cu128
CUDA available: True
CUDA device count: 1
GPU name: NVIDIA A100-SXM4-40GB
SECRET_WORD: I LOVE MMAI
```

Problem 2: Prepare your Dataset.

1. Explain some possible issues with converting non-image data into images (even if you did not have to do so, discuss what could be some issues).

In this assignment, I did not have to convert non-image data into images. In general, the transformation from non-image to image data adds an extra preprocessing step, increasing computational overhead without a guaranteed improvement in representational quality. In fact, the transformation is inherently *lossy*, as aspects of the original signal that resist visual encoding may be simply discarded. Moreover, the appearance of the data can depend on how the visualisation is created; choices such as scaling, color schemes, or smoothing can meaningfully change how patterns appear. Third, the conversion can introduce artifacts or biases absent from the original signal, which risk misleading the model into learning spurious patterns.

2. What are some possible issues with using visual representations of your data. Discuss some drawbacks of doing this (if you did not have to do the conversion as your data was already in the form of images, then discuss the drawbacks of converting those images to another modality like text, audio, etc.).

`CIFAR-10` images encode rich spatial information: textures, coloiurs and inter-object relationships that are difficult to *faithfully* represent in non-visual formats. A textual description, for instance, can capture broad semantic content but will typically omit subtle visual details that might matter for *fine-grained* classification. Converting to audio or similar representations would fare worse, as spatial structure has no natural analogue in such formats. In either case, the conversion reduces information available to the model and risks degrading its ability to learn meaningful visual patterns.

3. Discuss the strategy you decided on how to split your data into train/test splits. Why did you settle on this? Were any other alternative splits considered?

Our dataset was partitioned into an 80/20 train-test split, with random sampling to preserve class balance. The ratio was chosen to provide sufficient training data whilst maintaining a held-out set

large enough for reliable performance estimation.

We considered alternative strategies, namely, a 70/30 split or k-fold cross validation. The 80/20 split was ultimately preferred for its favourable balance between sample availabity and evaluation reliability, as well as its simplicity relative to cross-validation schemes.

Problem 3: Baseline Inference

After running the model on your four images, briefly discuss: What worked? (Which prompts or parameter settings produced better results?) What failed? (Were there recurring failure modes (e.g., hallucinations, vague answers)?) and patterns in mistakes (Did errors correlate with certain categories, lighting conditions, or question phrasing?)
We summarise the performance of the pre-trained `Qwen2.5-VL-3B-Instruct` model on four held-out `CIFAR-10` images in the table below.

Image ID	Question	Model Output	Ground Truth	Result
image_10.jpg	What object is shown in this image?	Map	deer	Incorrect
image_25.jpg	What object is shown in this image?	Turtle	frog	Incorrect
image_60.jpg	What object is shown in this image?	Car	automobile	Incorrect
image_120.jpg	What object is shown in this image?	Bird	bird	Correct

Table 1: Baseline model predictions on held-out `CIFAR-10` images.

The model achieved a single correct prediction, identifying the bird in `image_120.jpg`, suggesting it can handle relatively unambiguous subjects even at low resolution. The most notable failure mode was the production of out-of-vocabulary labels: 'Map' and 'Turtle' do not belong to the `CIFAR-10` category set, indicating the model drew on its broader visual vocabulary rather than reasoning within the restricted label space. The automobile misclassification is arguably the most forgivable, as 'Car' is semantically correct but does not match the dataset's exact label. Errors appeared to correlate with visual ambiguity introduced by the low resolution of `CIFAR-10` images (32×32 pixels), which obscures the fine-grained features needed for reliable identification, particularly, say, of small animals (e.g., deer, frog). Overall, the results suggest that, *without task-specific fine-tuning or label-constrained decoding*, the model's general visual understanding does not reliably transfer to this setting.

Problem 4: Prompt Engineering

Which types of prompt changes improved performance? Did adding context or structure help the model reason more effectively? Were there any surprising or inconsistent results?
The **baseline prompt** (*'You are a helpful assistant'*) produced mostly incorrect predictions, with the model frequently generating out-of-vocabulary labels, indicating reliance on general visual knowledge rather than the restricted label set. Explicitly constraining the output to the 10 `CIFAR-10` categories improves performance noticeably: the model produced valid labels and correctly identified 2/4 images, demonstrating that **label-restricted prompting** can meaningfully guide model behaviour. **Few-shot prompting** yielded comparable results, though the additional examples did not improve accuracy beyond what label restriction alone achieved. Counterintuitively, the **reasoning prompt** performed worst: encouraging the model to reason before answering appears to introduce additional variability, with the model producing more errors and generating further out-of-vocabulary labels (e.g., 'Dove'). Across all conditions, errors were most pronounced for visually similar categories (e.g., the model consistently confused deer and frog), whose distinguishing features are difficult to recover at 32×32 pixel resolution. Overall, **explicit output constraints** proved the most effective lever for adapting a general-purpose vision–language model to a restricted classification task without fine-tuning.

Problem 5: LoRA Fine-Tuning

Report the settings you used to get the best model. Which hyperparameters did you find have the most impact in the model's performance? Why do you think that is?

The best performing configuration achieved the lowest final training loss of 1.3076, with the following hyperparameters:

Hyperparameter	Value
Epochs	3
Learning Rate	1×10^{-4}
LoRA Rank (r)	4
LoRA Alpha	8
Batch Size	1
Gradient Accumulation	1
Image Resolution (Shortest Edge)	288
Max Sequence Length	512

Table 2: Hyperparameters for the best-performing LoRA fine-tuned model.

The hyperparameters with the greatest impact on performance where the number of epochs and LoRA rank. Increasing epochs from 3 to 5 did not improve training epochs and, in some cases slightly worsened it; similarly, increasing LoRA rank from 4 to 8 yielded no benefit.

Both hyperparameters govern how much the model adapts to the training data. With a relatively small dataset, training for too long risks overfitting or redundant updates that do not generalise. The LoRA rank controls the expressiveness of the low-rank adaptation matrices: whilst a higher rank increases representational capacity, it also introduces more trainable parameters, which can be counterproductive when data is limited. A lower rank combined with fewer epochs therefore produced the most favorable results for this task.

Problem 6: Post-Training Evaluation

Compare the pre-trained (in Step 2.2) and fine-tuned model outputs: Which questions showed improvement? Did LoRA fine-tuning correct any earlier mistakes? Were any new errors or biases introduced after fine-tuning? Document your observations and include examples where possible.

Image ID	Pretrained Output	Fine-Tuned Output	Ground Truth
image_10	Map	deer	deer
image_25	Turtle	frog	frog
image_60	Car	automobile	automobile
image_120	Bird	bird	bird

Table 3: Comparison of pretrained and fine-tuned model predictions on held-out `CIFAR-10` images.

After fine-tuning with LoRA, the same four held-out images were re-evaluated against the pre-trained baseline. 3/4 predictions improved: the pretrained model produced out-of-vocabularly labels (e.g., 'Map' for deer, 'Turtle' for frog) that LoRA fine-tuning corrected to 'deer', 'frog' and 'automobile', respectively. The bird image, already classified correctly by the pretrained model, remained correct after fine-tuning. No new errors were introduced, though the small evaluation set limits how broadly these improvements can be interpreted: testing on a larger held-out set would be necessary to assess generalisation more reliably. Overall, the resutls suggest that LoRA fine-tuning can effectively align a general-purpose vision–language model to a restricted label space with a relatively small number of trainable parameters.

Problem 7: Final Reflection

What concept did you find the most interesting?

The most compelling aspect of this assignment was **LoRA fine-tuning**. Rather than updating all parameters during fine-tuning, which is computationally prohibitive for large models, LoRA freezes the original weights and introduces small trainable update matrices alongside them. These updates are parameterised are the product of two low-rank matrices, meaning the essential adaptation is captured within a fraction of the parameters that full fine-tuning would require. This works because pretrained models already encode a vast amount of general knowledge; adapting to a new task requires only small, targeted adjustments rather than an overhaul of the entire network. The practical implication is significant: by training only a tiny fraction of the total parameters, LoRA makes fine-tuning large models feasible under limited computational resources.

Which concepts (if any) do you see being useful towards your goal? Why? If there was none, discuss why.

Several concepts connect directly to my work on the **EgoBlind-RA** project for MMAI, which focuses on assistive AI for egocentric visual understanding. Visual-language models and LoRA fine-tuning are particularly relevant: egocentric data is inherently domain-specific and often limited in scale, making parameter-efficient adaptation methods a practical necessity for specialising large pretrained models without prohibitive compute. The prompt design explanations were also instructive, demonstrating that careful instruction and output constraints can meaningfully shape model behaviour, which matters when the system must produce responses that are both reliable and interpretable for visually impaired users.

Is there a topic that was discussed during lectures up to the release of the assignment that you wished was covered in the homework? Any from the assignment that you wanted there to be touched upon more?

One topic worth exploring further is the systematic evaluation of vision-language models after fine-tuning; specifically, how fine-tuning on a narrow dataset like `CIFAR-10` affects generalization to unseen categories or slightly shifted image distributions. It would have also been interesting to explore interpretability and uncertainty estimation: understanding which parts of an image drive a prediction, and when the model is confident versus likely to err, is important for building systems that are not just accurate but also transparent and trustworthy. These considerations are especially relevant in high-stakes domains where blindly trusting model outputs is not an option.